

Vezérlési szerkezetek C++-ban: elágazás és ciklus

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Eddig a programjaid minden utasítást pontosan egyszer hajtottak végre, sorban. Most megtanulod, hogyan **döntsön** a program (elágazás), és hogyan **ismételjen** (ciklus). Ezzel a kettővel már igazi problémákat tudsz megoldani: a jegyzet végére prímszámokat fogsz keresni.

- **Próbáld ki, Tippelj, Kísérlet, Feladat** – ugyanúgy, mint az előző jegyzetben. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.
- **Ellenőrzés.** Minden feladatnál ott van, mit kell kapnod. Csak akkor lépj tovább, ha egyezik.
- **Hibakereső.** Ha valami nem működik, a jegyzet végén lévő táblázat segít.
- **Segítség.** A 7–9. fejezet nehezebb feladataihoz a jegyzet legvégén találsz útbaigazítást. Csak akkor lapozz oda, ha legalább öt percig próbálkoztál.

A jegyzet hosszú, és nem kell egy óra alatt a végére érned. Az 1–6. fejezet az alap; a 7–9. fejezet erre épít.

1. Döntés: az if

Az **if** utasítással a program csak akkor hajt végre valamit, ha egy feltétel teljesül.

Próbáld ki

```
#include <iostream>

int main() {
    int kor;
    std::cout << "Hany éves vagy? ";
    std::cin >> kor;

    if (kor >= 18) {
        std::cout << "Nagykoru vagy." << std::endl;
    }

    std::cout << "Viszlat!" << std::endl;
    return 0;
}
```

Futtasd kétszer: egyszer 20-at, egyszer 15-öt írf be. Melyik sor jelent meg mindkétszer, és melyik csak az egyik esetben?

- **if (kor >= 18)** – a zárójelben áll a **feltétel**. Ez egy kérdés, amelyre a válasz igaz vagy hamis.
- A kapcsos zárójelek közötti rész csak akkor fut le, ha a feltétel igaz.
- A záró kapcsos zárójel után a program mindenképp folytatódik.

A feltételekben ezeket az összehasonlító jeleket használhatod:

Jel	Jelentése	Példa
==	egyenlő	kor == 18
!=	nem egyenlő	kor != 18
< és >	kisebb, nagyobb	kor < 18
<= és >=	legfeljebb, legalább	kor >= 18

Tippelj

Legyen `x` értéke 7. Melyik sorok jelennek meg?

```
int x = 7;

if (x > 5) {
    std::cout << "A" << std::endl;
}
if (x == 5) {
    std::cout << "B" << std::endl;
}
if (x != 5) {
    std::cout << "C" << std::endl;
}
if (x <= 7) {
    std::cout << "D" << std::endl;
}
```

Kísérlet - a leggyakoribb hiba

Írd át az első program feltételét erre: `if (kor = 18)` – egyetlen egyenlőségjellel. Futtasd, és írd be 5-öt. Mit ír ki a program? Írasd ki a `kor` értékét is az `if` után!

Figyelem! Az összehasonlítás jele a **dupla** egyenlőségjel: `==`. Az egyszeres `=` értékadás: megváltoztatja a változót, és a feltétel szinte mindig igaz lesz. A fordító ezért nem jelez hibát!

Feladatok

1.1. feladat ★☆☆ - Pozitív

Kérj be egy egész számot. Ha pozitív, írd ki: `Pozitiv szam.`

Ellenőrzés: 5 → megjelenik a szöveg; -3 és 0 → nem jelenik meg.

1.2. feladat ★☆☆ - Titkos kód

A széf kódja 1234. Kérj be egy számot, és ha egyezik a kóddal, írd ki: `Kinyilt a szef.`

Ellenőrzés: 1234 → megjelenik; 1235 → nem jelenik meg.

1.3. feladat ★☆☆ - Páros

Kérj be egy egész számot. Ha páros, írd ki: `Paros`. **Tipp:** egy szám akkor páros, ha 2-vel osztva 0 a maradéka.

Ellenőrzés: 10 → megjelenik; 7 → nem jelenik meg.

1.4. feladat ★☆☆ - Kedvezmény

Kérd be egy vásárlás összegét. Ha eléri a 10 000 Ft-ot, a vásárló 10% kedvezményt kap. Írd ki a fizetendő összeget.

Ellenőrzés: 12000 → 10800; 8000 → 8000; 10000 → 9000.

2. Kétfelé és többfelé: else, else if

Az **else** ág akkor fut le, ha a feltétel hamis. Az **else if** segítségével több lehetőség közül választhatsz.

Próbáld ki - kétfelé

```
if (kor >= 18) {
    std::cout << "Nagykoru vagy." << std::endl;
} else {
    std::cout << "Kiskoru vagy." << std::endl;
}
```

A két ág közül mindig pontosan az egyik fut le.

Próbáld ki - többfelé

```
int pont;
std::cout << "Hany pontos lett a dolgozat? ";
std::cin >> pont;

if (pont >= 90) {
    std::cout << "jeles" << std::endl;
} else if (pont >= 75) {
    std::cout << "jo" << std::endl;
} else if (pont >= 60) {
    std::cout << "kozepes" << std::endl;
} else if (pont >= 40) {
    std::cout << "elegseges" << std::endl;
} else {
    std::cout << "elegtelen" << std::endl;
}
```

A program felülről lefelé vizsgálja a feltételeket, és az **első igaz** ágot hajtja végre. A többi már meg sem nézi. Próbáld ki 95, 80, 60 és 10 ponttal.

Tippelj - számít a sorrend?

Mit ír ki ez a változat 95 pontra? És miért?

```
if (pont >= 40) {
    std::cout << "elegseges" << std::endl;
} else if (pont >= 60) {
    std::cout << "kozepes" << std::endl;
} else if (pont >= 90) {
    std::cout << "jeles" << std::endl;
}
```

Feladatok

2.1. feladat ★☆☆ - Páros vagy páratlan

Kérj be egy egész számot, és írd ki, hogy páros vagy páratlan.

Ellenőrzés: 10 → **paros**; 7 → **paratlan**.

2.2. feladat ★☆☆ - A nagyobb

Kérj be két számot, és írd ki a nagyobbat. Ha egyenlők, ezt írd ki: **Egyenlok**.

Ellenőrzés: 3 és 8 → 8; 8 és 3 → 8; 5 és 5 → **Egyenlok**.

2.3. feladat ★☆☆ - Előjel

Kérj be egy számot, és írd ki, hogy pozitív, negatív vagy nulla.

Ellenőrzés: 5 → pozitív; -3 → negatív; 0 → nulla.

2.4. feladat ★☆☆ - Érdemjegy

Gépeld be a dolgozatos programot, és alakítsd át úgy, hogy a szöveg helyett a jegyet írja ki számmal.

Ellenőrzés: 95 → 5; 75 → 4; 74 → 3; 40 → 2; 12 → 1.

2.5. feladat ★☆☆ - Belépőjegy

Egy múzeumban 6 éves kor alatt ingyenes a belépés, 6 és 17 év között 1500 Ft, 18 és 64 év között 3000 Ft, 65 évtől 2000 Ft. Kérd be a látogató életkorát, és írd ki a jegy árát.

Ellenőrzés: 5 → 0; 6 → 1500; 17 → 1500; 18 → 3000; 64 → 3000; 65 → 2000.

2.6. feladat ★☆☆ - A legnagyobb a három közül

Kérj be három számot, és írd ki a legnagyobbat.

Ellenőrzés: 2, 9, 4 → 9; 9, 2, 4 → 9; 2, 4, 9 → 9; 5, 5, 1 → 5.

3. Összetett feltételek

Több feltételt is összekapcsolhatsz: az && jelentése „és”, a || jelentése „vagy”, a ! jelentése „nem”.

Próbáld ki

```
if (kor >= 13 && kor <= 19) {  
    std::cout << "Tinedzser vagy." << std::endl;  
}  
  
if (kor < 6 || kor >= 65) {  
    std::cout << "Ingyen utazhatsz." << std::endl;  
}
```

- Az && akkor igaz, ha **mindkét** oldala igaz.
- A || akkor igaz, ha **legalább az egyik** oldala igaz.
- A ! megfordítja a feltételt: a !(kor >= 18) ugyanaz, mint a kor < 18.

Tippelj - igaz vagy hamis?

Legyen a értéke 5, b értéke 10. Töltsd ki a középső oszlopot, aztán ellenőrizd: tedd a feltételt egy if zárójelébe, és nézd meg, lefut-e az ága.

Feltétel	A tippem	A program szerint
a > 3 && b > 3		
a > 7 && b > 7		
a > 7 b > 7		
a == 5 b == 5		
a != 5 && b == 10		
!(a > 7)		

Kísérlet - ahogy matematikából írnád

Matematikából így írnád: $13 \leq \text{kor} \leq 19$. Próbáld ki C++-ban is:

```
if (13 <= kor <= 19) {  
    std::cout << "Tinedzser vagy." << std::endl;  
}
```

Futtasd 50-nel. Mit ír ki?

Figyelem! A „két érték között” feltételt C++-ban mindig két összehasonlítással és **&&** jellel írd: **kor >= 13 && kor <= 19**. A matematikai írásmódot a fordító elfogadja, de nem azt jelenti, amit szeretnél!

Feladatok

3.1. feladat ★☆☆ - Határok között

Kérj be egy számot, és írd ki, hogy 1 és 10 közé esik-e (a határok is jók).

Ellenőrzés: 1 → igen; 10 → igen; 11 → nem; 0 → nem.

3.2. feladat ★☆☆ - Hétvége

Kérd be a nap sorszámát (1 = hétfő, ..., 7 = vasárnap). Írd ki, hogy hétköznap vagy hétvége. Ha a szám nem 1 és 7 közötti, ezt írd ki: **Nincs ilyen nap.**

Ellenőrzés: 3 → **hetkoznap**; 6 → **hetvege**; 7 → **hetvege**; 9 → **Nincs ilyen nap.**

3.3. feladat ★☆☆ - Mindkettővel osztható

Kérj be egy számot, és írd ki, hogy osztható-e 3-mal **és** 5-tel is.

Ellenőrzés: 15 → igen; 30 → igen; 9 → nem; 10 → nem.

3.4. feladat ★☆☆ - Háromszög

Kérd be három szakasz hosszát. Háromszög akkor szerkeszthető belőlük, ha bármelyik kettő összege nagyobb a harmadiknál. Írd ki, szerkeszthető-e.

Ellenőrzés: 3, 4, 5 → igen; 1, 2, 3 → nem; 2, 2, 5 → nem; 5, 5, 5 → igen.

3.5. feladat ★★ - Szökőév

Egy év szökőév, ha osztható 4-gyel; de ha 100-zal is osztható, akkor csak abban az esetben, ha 400-zal is. Kérj be egy évszámot, és írd ki, szökőév-e.

Ellenőrzés: 2024 → igen; 2023 → nem; 1900 → nem; 2000 → igen.

4. Ismétlés: a while ciklus

A ciklus újra és újra végrehajt egy programrészt. A **while** ciklus addig ismételi, amíg a feltétele igaz.

Próbáld ki

```
#include <iostream>

int main() {
    int i = 1;

    while (i <= 5) {
        std::cout << i << std::endl;
        i = i + 1;
    }

    std::cout << "Vege." << std::endl;
    return 0;
}
```

A **while** úgy néz ki, mint az **if**, de a kapcsos zárójelek közötti rész (a **ciklusmag**) nem egyszer fut le. A végén a program visszaugrik a feltételhez, és ha még mindig igaz, újra végrehajtja.

Kövesd végig papíron! Az első két sort kitöltöttem:

i értéke	Igaz-e, hogy i <= 5?	Mit ír ki?
1	igaz	1
2	igaz	2
3		
4		
5		
6		

Kísérlet - a ciklus, amely sosem áll meg

Töröld ki az **i = i + 1;** sort, és futtasd a programot. Mi történik? (A programot a Ctrl+C billentyűvel vagy az ablak bezárásával állíthatod meg.)

Figyelem! A ciklusmagban mindig változnia kell valaminek, amitől a feltétel egyszer hamis lesz. Különben a program soha nem áll meg: ezt hívjuk végtelen ciklusnak.

Tippelj

```
int i = 10;
while (i > 0) {
    std::cout << i << " ";
    i = i - 3;
}
```

A növelést és a csökkentést rövidebben is írhatod: az **i++** ugyanaz, mint az **i = i + 1**; az **i--** ugyanaz, mint az **i = i - 1**; az **i += 3** pedig ugyanaz, mint az **i = i + 3**.

Próbáld ki - addig kérdez, amíg jó választ nem kap

```
int szam;
std::cout << "Írj be egy pozitív számot: ";
std::cin >> szam;

while (szam <= 0) {
    std::cout << "Ez nem pozitív. Próbáld újra: ";
    std::cin >> szam;
}

std::cout << "Köszönöm: " << szam << std::endl;
```

Itt nem tudjuk előre, hányszor fut le a ciklus: lehet, hogy egyszer sem, lehet, hogy tízszer.

Feladatok

4.1. feladat ★☆☆ - Tízíg

Írd ki a számokat 1-től 10-ig egy sorba, szóközzel elválasztva.

Ellenőrzés: 1 2 3 4 5 6 7 8 9 10

4.2. feladat ★☆☆ - Visszaszámlálás

Számolj vissza 10-től 1-ig, a végén írd ki: **Start!**

Ellenőrzés: 10 9 8 7 6 5 4 3 2 1 Start!

4.3. feladat ★☆☆ - Páros számok

Írd ki a páros számokat 2-től 20-ig.

Ellenőrzés: 2 4 6 8 10 12 14 16 18 20

4.4. feladat ★☆☆ - Összeg nulláig

Kérj be számokat egészen addig, amíg a felhasználó 0-t nem ír be. Írd ki a beírt számok összegét és azt, hány szám volt (a nullát ne számold bele).

Ellenőrzés: 5, 8, 2, 0 → az összeg 15, 3 szám volt.

4.5. feladat ★☆☆ - Kódtörés

A széf kódja 1234. Addig kérj be számokat, amíg a felhasználó el nem találja. Írd ki, hányadik próbálkozásra sikerült.

Ellenőrzés: 1, 99, 1234 → 3. próbálkozásra sikerült.

4.6. feladat ★☆☆ - Duplázás

Indulj az 1-ből, és duplázd addig, amíg meg nem haladja az 1000-et. Hány duplázás kellett, és mennyi lett a szám?

Ellenőrzés: 10 duplázás, a szám 1024.

4.7. feladat ★★★ - Számjegyek

Kérj be egy pozitív egész számot. Írd ki, hány számjegyből áll, és mennyi a számjegyeinek összege.

Tipp: az utolsó számjegyet az $n \% 10$ adja meg; az $n / 10$ pedig levágja az utolsó számjegyet.

Ellenőrzés: 2026 → 4 számjegy, az összegük 10; 7 → 1 számjegy, az összeg 7; 99999 → 5 számjegy, az összeg 45.

5. A for ciklus

Ha előre tudod, hányszor kell ismételni, a **for** ciklus kényelmesebb: a számláló kezelése egyetlen sorba kerül.

Próbáld ki

```
for (int i = 1; i <= 5; i++) {  
    std::cout << i << std::endl;  
}
```

Ez pontosan ugyanazt csinálja, mint az előző fejezet első programja. A zárójelben három rész áll, pontosvesszővel elválasztva:

- **int i = 1** – a **kezdőérték**: egyszer hajtódik végre, a ciklus előtt.
- **i <= 5** – a **feltétel**: amíg igaz, a ciklusmag lefut.
- **i++** – a **léptetés**: minden kör végén végrehajtódik.

Tippelj – for-totó

Mit írnak ki ezek a ciklusok, ha a ciklusmagban mindig az **i** értékét írjuk ki? Töltsd ki, aztán ellenőrizd.

A ciklus feje	A tippem	A program szerint
<code>for (int i = 0; i < 5; i++)</code>		
<code>for (int i = 2; i <= 10; i += 2)</code>		
<code>for (int i = 10; i > 0; i -= 3)</code>		
<code>for (int i = 1; i <= 100; i *= 2)</code>		
<code>for (int i = 5; i < 5; i++)</code>		

Próbáld ki – gyűjtögetés a ciklusban

A ciklus közben gyűjthetsz is. **Összegzés** – a számok összege 1-től 100-ig:

```
int osszeg = 0;  
for (int i = 1; i <= 100; i++) {  
    osszeg = osszeg + i;  
}  
std::cout << osszeg << std::endl;
```

Az **osszeg** olyan, mint egy persely: üresen (0-ról) indul, és minden körben beledobjuk a következő számot.

Megszámlálás – a 7-tel osztható számok darabszáma 1 és 100 között:

```
int db = 0;  
for (int i = 1; i <= 100; i++) {  
    if (i % 7 == 0) {  
        db++;  
    }  
}  
std::cout << db << std::endl;
```

A legnagyobb – öt beolvasott szám közül a legnagyobb:

```
int legnagyobb;
std::cin >> legnagyobb;

for (int i = 2; i <= 5; i++) {
    int x;
    std::cin >> x;
    if (x > legnagyobb) {
        legnagyobb = x;
    }
}
std::cout << "A legnagyobb: " << legnagyobb << std::endl;
```

Az első számot kinevezzük „eddigi legnagyobbnak”, a többi pedig sorban összehasonlítjuk vele.

Kísérlet - hol legyen a persely?

Tedd át az összegző programban az `int osszeg = 0;` sort a ciklus belsejébe. Lefordul? Ha a kiírást is beteszed a ciklusba, mit láatsz?

Feladatok

5.1. feladat ★☆☆ - Négyzetszámok

Írd ki 1-től 10-ig a számokat és a négyzetüket, soronként egyet.

Ellenőrzés: az első sor `1 1`, a harmadik `3 9`, az utolsó `10 100`.

5.2. feladat ★☆☆ - Szorzótábla

Kérj be egy számot, és írd ki a szorzótábláját 1-től 10-ig.

Ellenőrzés: 7 esetén az első sor `1 x 7 = 7`, az utolsó `10 x 7 = 70`.

5.3. feladat ★☆☆ - Összeg

Kérj be egy `n` számot, és írd ki a számok összegét 1-től `n`-ig.

Ellenőrzés: `10 → 55`; `100 → 5050`; `1 → 1`.

5.4. feladat ★☆☆ - Faktoriális

Kérj be egy `n` számot, és írd ki a számok szorzatát 1-től `n`-ig. **Tipp:** milyen értékről induljon a gyűjtő változó, ha szorzol?

Ellenőrzés: `5 → 120`; `10 → 3628800`; `1 → 1`.

5.5. feladat ★☆☆ - Hárommal oszthatók

Hány 3-mal osztható szám van 1 és 100 között, és mennyi az összegük?

Ellenőrzés: 33 darab, az összegük 1683.

5.6. feladat ★☆☆ - Átlag

Kérdezd meg, hány számot ad meg a felhasználó, olvasd be őket, és írd ki az átlagukat.

Ellenőrzés: 4 szám (5, 3, 4, 5) → 4.25.

5.7. feladat ★☆☆ - Legkisebb és legnagyobb

Kérj be öt számot, és írd ki a legkisebbet és a legnagyobbat.

Ellenőrzés: 4, -2, 9, 0, 7 → a legkisebb -2, a legnagyobb 9.

5.8. feladat ★★ - Hatvány

Kérj be egy alapot és egy kitevőt, és számold ki a hatványt ciklussal.

Ellenőrzés: 2 és 10 → 1024; 3 és 4 → 81; 5 és 0 → 1.

6. Ciklus és elágazás együtt: osztók

Ha a ciklus belsejébe elágazást teszel, a számok közül válogatni tudsz. Ebből lesz később a prímkeresés.

Próbáld ki

Egy szám osztói azok a számok, amelyekkel maradék nélkül osztható. Ez a program kiírja a beolvasott szám összes osztóját:

```
int n;
std::cout << "Melyik szám osztóit keresed? ";
std::cin >> n;

for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
        std::cout << i << " ";
    }
}
std::cout << std::endl;
```

A ciklus végigpróbálja a számokat 1-től **n**-ig, és amelyekkel az **n** maradék nélkül osztható, azt kiírja. Próbáld ki 12-vel, 7-tel és 36-tal.

Feladatok

6.1. feladat ★☆☆ - Osztók száma

Alakítsd át a programot: ne kiírja, hanem megszámlolja az osztókat.

Ellenőrzés: 12 → 6 osztó; 7 → 2 osztó; 36 → 9 osztó; 1 → 1 osztó.

6.2. feladat ★☆☆ - Osztók összege

Számold ki a beolvasott szám osztóinak összegét.

Ellenőrzés: 12 → 28; 7 → 8; 36 → 91.

6.3. feladat ★☆☆ - FizzBuzz

Írd ki a számokat 1-től 30-ig, soronként egyet. De a 3-mal osztható számok helyett **Fizz**, az 5-tel oszthatók helyett **Buzz**, a mindkettővel oszthatók helyett **FizzBuzz** álljon.

Ellenőrzés: a sorok eleje: 1, 2, Fizz, 4, Buzz, Fizz, 7. A 15. sor FizzBuzz. Összesen 8 sorban áll Fizz, 4 sorban Buzz, 2 sorban FizzBuzz.

6.4. feladat ★☆☆ - Közös osztók

Kérj be két számot, és írd ki a közös osztóikat. Írd ki a legnagyobbat is külön.

Ellenőrzés: 12 és 18 → a közös osztók 1 2 3 6, a legnagyobb 6.

7. Egymásba ágyazott ciklusok

Egy ciklus belsejébe másik ciklust is írhatasz. Ilyenkor a belső ciklus a külső **minden egyes** körében elejétől a végéig lefut.

Próbáld ki

```
for (int sor = 1; sor <= 3; sor++) {
    for (int oszlop = 1; oszlop <= 5; oszlop++) {
        std::cout << "*";
    }
    std::cout << std::endl;
}
```

A külső ciklus a sorokat számolja, a belső egy soron belül a csillagokat. Hány csillagot ír ki a program összesen?

Tippelj - milyen sorrendben?

Mit ír ki ez a program? Írd le a füzetedbe mind a hat sort, mielőtt futtatod.

```
for (int a = 1; a <= 2; a++) {  
    for (int b = 1; b <= 3; b++) {  
        std::cout << a << " " << b << std::endl;  
    }  
}
```

Figyeld meg: a külső változó (a) lassan változik, a belső (b) gyorsan – mint az óra kismutatója és nagymutatója.

Próbáld ki - a belső ciklus hossza változik

Írd át az első program belső ciklusának feltételét erre: `oszlop <= sor`. Mit rajzol most a program? Miért?

Feladatok

7.1. feladat ★☆☆ - Téglalap

Kérd be egy téglalap szélességét és magasságát, és rajzold ki csillagokból.

Ellenőrzés: 6 és 2 esetén két sor, mindegyikben hat csillag.

7.2. feladat ★☆☆ - Háromszög

Kérj be egy `n` számot, és rajzolj `n` soros háromszöget.

Ellenőrzés: 4 esetén:

```
*  
**  
***  
****
```

7.3. feladat ★☆☆ - Fordított háromszög

Rajzold meg a háromszöget fejjel lefelé.

Ellenőrzés: 4 esetén:

```
****  
***  
**  
*
```

7.4. feladat ★☆☆ - Számháromszög

Csillagok helyett számokat írf ki.

Ellenőrzés: 4 esetén:

```
1  
1 2  
1 2 3  
1 2 3 4
```

7.5. feladat ★★☆☆ - Szorzótábla

Írd ki az 5×5 -ös szorzótáblát táblázatként. **Tipp:** a számok után "\t" (tabulátor) kiírásával szép oszlopokat kapsz.

Ellenőrzés: öt sor; az első 1 2 3 4 5, az utolsó 5 10 15 20 25.

7.6. feladat ★★☆☆ - Dobókockák

Két dobókockával dobunk. Írd ki az összes olyan dobást, amelyben a két szám összege 7. Hány ilyen van? És hány olyan, ahol az összeg 10?

Ellenőrzés: 7 az összeg 6 esetben (1 6, 2 5, 3 4, 4 3, 5 2, 6 1); 10 az összeg 3 esetben.

7.7. feladat ★★☆☆ - Piramis

Rajzolj n soros piramist. A sorok elejére szóközök is kellenek!

Ellenőrzés: 4 esetén:

```
  *
 ***
*****
*****
```

7.8. feladat ★★☆☆ - Sakktábla

Rajzolj 8×8 -as sakktáblát: a sötét mezők helyén #, a világosak helyén . álljon, a bal felső sarok legyen #.

Ellenőrzés: az első sor #.#.#.#., a második .#.#.#.#, és így váltakoznak.

8. Prímszámok

Egy szám akkor **prím**, ha pontosan két osztója van: az 1 és önmaga. A 7 prím (osztói: 1, 7), a 9 nem az (osztói: 1, 3, 9), és az 1 sem prím (egyetlen osztója van).

Ebben a fejezetben lépésről lépésre építesz fel egy prímkereső programot. Minden feladat az előzőre épül.

8.1. feladat ★☆☆☆ - Prím-e? Első változat

A 6.1. feladatban megszámoltad egy szám osztóit. Ennek alapján írd programot, amely egy beolvasott számról megmondja, prím-e.

Ellenőrzés: 7 → prím; 2 → prím; 9 → nem prím; 1 → nem prím; 97 → prím.

Próbáld ki - jelző változóval

Az osztók megszámlálása helyett van egyszerűbb gondolat is: egy szám akkor **nem** prím, ha 2 és önmaga között van osztója. Elég tehát egy **bool** típusú változó, amely megjegyzi, találtunk-e ilyet:

```
int n;
std::cin >> n;

bool prim = true;
if (n < 2) {
    prim = false;
}
for (int i = 2; i < n; i++) {
    if (n % i == 0) {
        prim = false;
    }
}

if (prim) {
    std::cout << n << " prim." << std::endl;
} else {
    std::cout << n << " nem prim." << std::endl;
}
```

A **prim** változó igazról indul: „ártatlan, amíg be nem bizonyosodik az ellenkezője”. Ha a ciklus talál egy osztót, hamisra állítja. A ciklus **után** már csak meg kell néznünk, mi maradt benne.

8.2. feladat ★★☆☆ - Prímek százig

Írd ki az összes prímszámot 2-től 100-ig. **Tipp:** a fenti vizsgálat kerüljön egy külső ciklus belsejébe, amely az **n** értékét lépteti 2-től 100-ig.

Ellenőrzés: a sor így kezdődik: **2 3 5 7 11 13**, és 97-tel végződik.

8.3. feladat ★★☆☆ - Hány prím van?

Számold meg, hány prímszám van 100-ig, és hány 1000-ig.

Ellenőrzés: 100-ig 25, 1000-ig 168.

8.4. feladat ★★☆☆ - A következő prím

Kérj be egy számot, és írd ki a nála nagyobb legkisebb prímszámot.

Ellenőrzés: 20 → 23; 13 → 17; 100 → 101.

8.5. feladat ★★☆☆ - Ikerprímek

Ikerprímeknek nevezzük az olyan prímpárokat, amelyek különbsége 2, például a 11 és a 13. Írd ki az összes ikerprímpárt 100-ig.

Ellenőrzés: 8 pár: (3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73).

8.6. feladat ★★☆☆ - Gyorsabban!

A prímvizsgálatnál nem kell **n**-ig próbálkozni: elég addig menni, amíg **i * i <= n**. Gondold végig, miért! (Ha a 36-nak a 9 osztója, akkor a 4 is az: az osztók párban járnak.) Írd át a programodat, és számold meg mindkét változatban, hány osztást végez a 9973 vizsgálatakor.

Ellenőrzés: a 9973 prím. A régi változat 9971, az új 98 osztást végez.

8.7. feladat ★★☆☆ - Prímtényezős felbontás

Kérj be egy számot, és írd ki a prímtényezős felbontását.

Ellenőrzés: 360 → **2 2 2 3 3 5**; 84 → **2 2 3 7**; 97 → **97**; 1001 → **7 11 13**.

9. Kihívások

Ezekben a feladatokban már neked kell kitalálnod, hogyan rakd össze az eddig tanultakat. Válaszd azt, amelyik megtetszik. Ha elakadsz, a jegyzet végén találsz segítséget.

9.1. feladat ★★☆☆ - Fibonacci-sorozat

A sorozat első két tagja 1 és 1, minden további tag az előző kettő összege. Írd ki az első 15 tagot.

Ellenőrzés: 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610

9.2. feladat ★★☆☆ - A $3n + 1$ probléma

Válassz egy pozitív egész számot. Ha páros, oszd el kettővel; ha páratlan, szorozd meg hárommal, és adj hozzá egyet. Ismételd, amíg 1-et nem kapsz. Írd ki a sorozatot, és azt, hány lépésből állt.

Ellenőrzés: $6 \rightarrow 6 \ 3 \ 10 \ 5 \ 16 \ 8 \ 4 \ 2 \ 1$, 8 lépés. A 27-ből indulva 111 lépés kell.

9.3. feladat ★★☆☆ - Fordított szám

Kérj be egy pozitív egész számot, és állítsd elő a „megfordítását” egy másik változóban. Írd ki azt is, hogy a szám visszafelé olvasva ugyanaz-e.

Ellenőrzés: $1234 \rightarrow 4321$, nem ugyanaz; $12321 \rightarrow 12321$, ugyanaz.

9.4. feladat ★★☆☆ - Legnagyobb közös osztó

A 6.4. feladatban végigpróbáltad az összes számot. Az ókori görögök ennél sokkal gyorsabb módszert ismertek: amíg a b nem nulla, számold ki az a és a b osztási maradékát; az a új értéke legyen a b , a b új értéke a maradék. A végén az a a legnagyobb közös osztó.

Ellenőrzés: 48 és 18 $\rightarrow$ 6; 100 és 75 $\rightarrow$ 25; 17 és 5 $\rightarrow$ 1.

9.5. feladat ★★☆☆ - Tökéletes számok

Egy szám tökéletes, ha megegyezik a nála kisebb osztóinak összegével: $6 = 1 + 2 + 3$. Keresd meg az összes tökéletes számot 1000-ig.

Ellenőrzés: három ilyen szám van: 6, 28, 496.

9.6. feladat ★★☆☆ - Armstrong-számok

A 153 különleges szám: $1 \cdot 1 \cdot 1 + 5 \cdot 5 \cdot 5 + 3 \cdot 3 \cdot 3 = 153$. Keresd meg az összes háromjegyű számot, amely megegyezik a számjegyei köbének összegével.

Ellenőrzés: négy ilyen szám van: 153, 370, 371, 407.

9.7. feladat ★★☆☆ - Pitagoraszai számhármak

Keresd meg az összes olyan a, b, c egész számhármast, amelyre $a < b < c$, egyik sem nagyobb 20-nál, és $a \cdot a + b \cdot b = c \cdot c$.

Ellenőrzés: hat hármak van: (3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15), (12, 16, 20).

9.8. feladat ★★☆☆ - Goldbach-sejtés

A sejtés szerint minden 2-nél nagyobb páros szám felírható két prímszám összegeként. Kérj be egy páros számot, és írd ki azt a felbontását, amelyben az első prím a lehető legkisebb.

Ellenőrzés: $28 \rightarrow 5 + 23$; $100 \rightarrow 3 + 97$; $4 \rightarrow 2 + 2$.

9.9. feladat ★★☆☆ - Az n -edik prím

Kérj be egy n számot, és írd ki az n -edik prímszámot.

Ellenőrzés: az 1. prím a 2; a 10. prím a 29; a 100. prím az 541.

Hibakereső: mi lehet a baj?

Az elágazások és ciklusok hibái gyakran nem járnak hibaüzenettel: a program lefut, csak mást csinál, mint amit szeretnél. Keresd meg a táblázatban, amit tapasztalsz.

Amit látsz	Mit jelent?	Mit tegyél?
Az <code>if</code> ága mindig lefut, bármit írsz be	Egyszeres <code>=</code> áll a feltételben, vagy pontosvessző került az <code>if</code> sorának végére.	Összehasonlításhoz <code>==</code> kell. Az <code>if (...)</code> után ne legyen pontosvessző.
A program nem áll meg, vég nélkül ír vagy „lefagy”	Végtelen ciklus: a feltétel soha nem lesz hamis.	Állítsd le (Ctrl+C). Változik a ciklusváltozó? Jó irányba?
A ciklus egyszer sem fut le	A feltétel már az első vizsgálatkor hamis.	Nézd meg a kezdőértéket és a relációjel irányát.
Eggyel több vagy eggyel kevesebb kör fut le	A feltételben <code><</code> áll <code><=</code> helyett, vagy fordítva.	Kövesd végig papíron az utolsó kört.
Az összeg vagy a darabszám hibás (például mindig 0 vagy az utolsó érték)	A gyűjtő változót a cikluson belül hoztad létre, így minden körben újraindul.	Az <code>int osszeg = 0;</code> kerüljön a ciklus elé.
<code>'i' was not declared in this scope</code>	A <code>for</code> ciklus változója csak a cikluson belül létezik.	Ha a ciklus után is kell, hozd létre a ciklus előtt.
<code>'else' without a previous 'if'</code>	Az <code>else</code> előtt hiányzik az <code>if</code> ág lezárása.	Ellenőrizd a kapcsos zárójeleket. Nincs pontosvessző az <code>if (...)</code> után?
<code>expected '}' at end of input</code>	Több nyitó kapcsos zárójel van, mint záró.	Számold össze a párokat; a szép behúzás sokat segít.
A „két érték között” feltétel mindig igaz	Matematikai írásmódot használtál: <code>1 <= x <= 10</code> .	<code>x >= 1 && x <= 10</code>
Az alakzat egyetlen hosszú sorba kerül	Hiányzik a sorvége a belső ciklus után.	A <code>std::cout << std::endl;</code> a belső ciklus után, de a külsőn belül legyen.
Minden csillag külön sorba kerül	A sorvége a belső ciklusba került.	Tedd a belső ciklus záró kapcsos zárójele mögé.

Összefoglaló táblázat (puska)

Minta	Jelentés
<code>if (feltétel) { ... }</code>	csak akkor fut le, ha a feltétel igaz
<code>if (...) { ... } else { ... }</code>	kétfelé ágazás
<code>if (...) { ... } else if (...) { ... } else { ... }</code>	többfelé ágazás: az első igaz ág fut le
<code>==, !=, <, >, <=, >=</code>	összehasonlítás
<code>&&, , !</code>	és, vagy, nem
<code>while (feltétel) { ... }</code>	ismétlés, amíg a feltétel igaz
<code>for (int i = 1; i <= 10; i++) { ... }</code>	számlálós ciklus 1-től 10-ig
<code>i++, i--, i += 2</code>	növelés eggyel, csökkentés eggyel, növelés kettővel
<code>int osszeg = 0;</code> a ciklus előtt	gyűjtő változó összegzéshez
<code>bool van = false;</code> a ciklus előtt	jelző változó eldöntéshez
<code>n % i == 0</code>	az <code>n</code> osztható <code>i</code> -vel

Figyelem! Három dolog, amit soha ne felejts el: az összehasonlítás jele a dupla egyenlőségjel; a ciklusban mindig változzon valami, különben nem áll meg; a gyűjtő és a jelző változókat a ciklus **előtt** hozd létre.

Segítség a feladatokhoz

Csak akkor olvasd el egy feladat segítségét, ha már legalább öt perccig próbálkoztál vele. A segítség nem megoldás: azt mondja meg, merre indulj.

4-6. fejezet

- **4.7.** `while (n > 0)` ciklus: minden körben add hozzá az összeghez az `n % 10` értékét, növekd a számlálót, és oszd el az `n`-et tízzel.
- **5.4.** Szorzásnál a gyűjtő változó 1-ről induljon, ne 0-ról.
- **5.8.** A kitevő azt mondja meg, hányszor kell az eredményt megszorozni az alappal. Az eredmény 1-ről induljon.
- **6.3.** Először a „mindkettővel osztható” esetet vizsgálj, különben soha nem jut rá sor (emlékezz a 2. fejezet sorrendes kísérletére).
- **6.4.** A ciklus menjen 1-től a kisebbik számig; a feltétel: mindkét számot osztja. A legnagyobb közös osztó az utoljára talált közös osztó.

7. fejezet

- **7.3.** Kétféle megoldás van: a külső ciklus számolhat visszafelé (`sor = n-től 1-ig`), vagy a belső ciklus hossza lehet `n - sor + 1`.
- **7.4.** A belső ciklus változóját írd ki, ne csillagot.
- **7.5.** A kiírandó érték a két ciklusváltozó szorzata.
- **7.6.** Két egymásba ágyazott ciklus 1-től 6-ig, bennük egy `if` az összegre és egy számláló.
- **7.7.** Minden sorban két belső ciklus kell egymás után: az első szóközöket ír (`n - sor` darabot), a második csillagokat (`2 * sor - 1` darabot).
- **7.8.** Egy mező sötét, ha a sor és az oszlop sorszámának összege páros.

8. fejezet

- **8.1.** Számold meg az osztókat (6.1. feladat), és a ciklus után vizsgálj meg: ha pontosan 2 van, a szám prím.
- **8.2.** A külső ciklus: `for (int n = 2; n <= 100; n++)`. A belsejébe másold be a jelző változós vizsgálatot. Fontos: a `bool prim = true`; sor a külső ciklus **belsejében** legyen, hogy minden számnál újra igazról induljon.
- **8.3.** Ugyanaz, mint a 8.2., de kiírás helyett számlálót növelsz.
- **8.4.** Vizsgálj sorban a `sza + 1`, `sza + 2`, ... számokat, amíg prímet nem találsz. Ehhez `while` ciklus kell, amelynek a feltétele egy jelző változó: „még nem találtam”.
- **8.5.** Jegyezd meg az előző megtalált prímet. Ha az új prím pontosan 2-vel nagyobb nála, találtál egy párt.
- **8.6.** A belső ciklus feltétele `i < n` helyett `i * i <= n`. Az osztások megszámlálásához tegyél egy számlálót a belső ciklusba.
- **8.7.** Indulj a `p = 2` osztóval. Amíg az `n` nagyobb 1-nél: ha `n` osztható `p`-vel, írd ki a `p`-t, és oszd el vele az `n`-et (a `p` marad!); ha nem osztható, növekd a `p`-t.

9. fejezet

- **9.1.** Két változóban tartsd az utolsó két tagot (**elozo** és **mostani**). Minden körben számold ki az összegüket egy harmadik változóba, aztán „léptesd” a kettőt.
- **9.2.** **while (n != 1)** ciklus, benne **if - else** a páros és a páratlan esetre, és egy lépésszámláló.
- **9.3.** A 4.7. feladat ciklusa kell. A megfordított szám építése: **forditott = forditott * 10 + n % 10**. Az eredeti számot tedd el egy másik változóba, mert a ciklus „elfogyasztja”.
- **9.4.** **while (b != 0)** ciklus. Kell egy segédváltozó a maradéknak, mert az **a** és a **b** értéke egyszerre változik.
- **9.5.** Külső ciklus a számokra 2-től 1000-ig, belső ciklus az osztók összegére (6.2. feladat, de a számot magát ne add hozzá).
- **9.6.** Ciklus 100-tól 999-ig. A három számjegy: **n / 100**, **n / 10 % 10** és **n % 10**.
- **9.7.** Három egymásba ágyazott ciklus. Ha a **b** ciklusa **a + 1**-ről, a **c** ciklusa **b + 1**-ről indul, az **a < b < c** feltétel magától teljesül.
- **9.8.** Próbáld végig a **p** értékeit 2-től: az első olyan **p** a megoldás, amelyre a **p** és az **n - p** is prím. Két prímvizsgálat kell egymás után, két külön jelző változóval.
- **9.9.** Számold a megtalált prímeket egy **while** ciklusban, amíg a darabszám el nem éri az **n**-et.